

基于 MATLAB 的典型理想滤波电路传递函数与瞬态响应仿真

赵楼略

(微电子科学与工程)

指导教师: 郑斌、赵纪伟

摘要 本文设计并实现了一套基于 MATLAB 的滤波电路仿真系统。系统以拉普拉斯变换和传递函数为理论基础, 支持典型无源 RC、RL、RLC 滤波器及理想运放有源滤波器的参数化分析, 并可读取常见 EDA 软件导出的 RLC 网表, 自动识别元件、节点与输入输出端口。程序能够计算系统函数、幅频与相频响应、零极点、单位阶跃和单位冲激响应, 并利用符号运算求取自定义输入下的时域表达式; 同时通过图形界面完成电路选择、网表可视化和结果展示。以二阶无源低通和 Sallen-Key 有源带阻电路为例进行理论计算与仿真验证, 结果表明程序所得频率特性与理论分析及 Multisim 仿真基本一致。

关键词 MATLAB; 滤波电路; 传递函数; 瞬态响应; 信号与系统

一、问题的提出

本课程设计选择题目 8: “小型的系统仿真”, 并选择“典型滤波电路传递函数与瞬态响应”这一方向进行仿真探究, 旨在利用信号与系统课程中所学的拉普拉斯变换等知识搭建一个面向实际滤波电路的 Matlab 仿真系统。

滤波电路为模拟电路设计的一个重要方向, 在电子电路设计中起到不可或缺的作用, 因而对各类滤波器的电路仿真工具软件应运而生。目前较成熟的仿真工具如 Pspice、LTspice 等更偏向实际工程的应用, 其强烈依赖于较为准确的元器件 SPICE 模型(通过后期测量建模得到), 基于节点分析、数值积分进行仿真求解, 并不能得到表达式的解析解。

因此想到, 能不能用 Matlab 做一个纯基于数学与信号理论的理想滤波电路仿真器, 基于电阻、电容等元器件 s 域模型以及拉普拉斯变换、拉普拉斯反变换计算传递函数、瞬态响应, 并配合 gui 等辅助设计, 实现数学上严谨、工程上实用并且人机交互友好等指标。

该项目与信号与系统课程关联紧密: 电路基本元器件的 s 域模型、传递函数的计算方法以及由 s 域信号转成时域信号的拉普拉斯逆变换全都来源于课内; 以及“滤波器”在信号与系统中作为一类系统也有重要应用; 综上, 研究滤波电路的仿真系统十分贴合信号与系统的课程主题, 也较有研究意义。

二、解决问题的原理、技术方案与算法

2.1 基本原理

2.1.1 滤波电路的系统函数

滤波电路本质上可以看作一个连续时间线性时不变系统。对于滤波电路已知输入求解输出的问题, 若直接通过时域的方式去算, 则会计算复杂、计算量大; 因此在工程上通常采用拉普拉斯变换, 将其转成 s 域后求解系统函数, 再经逆变换求输出, 常见的求解形式为:

$$x(t) \xrightarrow{\mathcal{L}} X(s) \xrightarrow{\times H(s)} Y(s) \xrightarrow{\mathcal{L}^{-1}} y(t)$$

对于零初始条件下的单输入单输出电路, 若输入为 $x(t)$ 、输出为 $y(t)$, 其拉普拉斯变换分别为 $X(s)$ 和 $Y(s)$, 则系统传递函数定义为

$$H(s) = \frac{Y(s)}{X(s)}.$$

传递函数只由电路结构和元器件参数决定, 与具体输入信号无关。因此, 在完成电路到 $H(s)$ 的建模后, 系统的频域特性、零极点分布以及对任意输入的时域响应均可由同一个数学模型得到。

2.1.2 无源元器件的 s 域模型

无源滤波电路的建模采用元器件的 s 域等效阻抗。电阻、电感、电容分别有

$$Z_R = R, Z_L = sL, Z_C = \frac{1}{sC}.$$

在此基础上，利用串并联阻抗关系和分压关系即可得到典型 RC、RL、RLC 滤波器的传递函数。例如一阶 RC 低通滤波器输出取电容两端（如图1），有

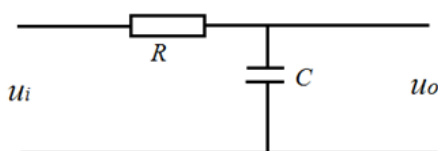


图1 一阶无源低通滤波器

$$H(s) = \frac{Z_C}{R + Z_C} = \frac{1}{RCs + 1}.$$

类似地，高通、带通、带阻等结构可通过选择不同输出端口或不同串并联网路得到。二阶电路的分母通常可化为

$$s^2 + \frac{\omega_0}{Q}s + \omega_0^2,$$

其中 ω_0 为固有角频率， Q 为品质因数，它们共同决定谐振位置、带宽和过渡带形状。

2.1.3 频率响应与波特图

频率响应由传递函数在虚轴上的取值得到，即令 $s = j\omega$,

$$H(j\omega) = H(s)|_{s=j\omega}.$$

幅频特性通常表示为 $20 \log_{10} |H(j\omega)|$ ，相频特性表示为 $\angle H(j\omega)$ 。波特图正是对这两个量随频率变化的图形化描述。因此，程序只需在给定频率范围内对 $H(j\omega)$ 进行数值采样，便可绘制幅频、相频及完整波特图。

2.1.4 瞬态响应

时域响应通过拉普拉斯反变换获得。单位冲激响应为

$$h(t) = \mathcal{L}^{-1}\{H(s)\},$$

单位阶跃响应为

$$y_{\text{step}}(t) = \mathcal{L}^{-1}\left\{\frac{H(s)}{s}\right\}.$$

若给定一般输入 $x(t)$ ，程序先计算其拉普拉斯变换 $X(s)$ ，再由

$$Y(s) = H(s)X(s), y(t) = \mathcal{L}^{-1}\{Y(s)\}$$

求出输出的时域表达式。这样既能得到数值曲线，也能方便地展示解析表达式。

2.1.5 理想运放有源滤波器

对于有源滤波器，本文采用理想运放模型：输入阻抗无穷大，输出阻抗为零，开环增益足够大，且电源轨足够高，不考虑饱和、压摆率和增益带宽积限制。在该假设下，运放主要提供受控源和反馈约束，滤波特性仍由外部电阻电容网络决定。典型的一阶有源滤波器可看作 RC 网络后接理想放大环节；二阶 Sallen-Key 及多反馈结构则可归一化为二阶系统模型。该处理方式忽略了非理想器件误差，但便于突出拉普拉斯变换和传递函数分析的课程重点。

2.2 实现算法

2.2.1 总体计算流程设计

程序的整体流程按照“输入电路结构——建立传递函数——计算响应——显示结果”展开。

输入部分中，程序的 gui 可供用户选择典型的滤波电路模型并设置电阻电容值，也可以直接读取本地的电路网表文件进行电路分析（目前只支持无源 RLC 滤波电路）。前者由程序内置的模型库给出传递函数，后者先解析网表中的元器件和节点，再尝试转化为可计算的滤波器结构。二者最终都统一为分子、分母多项式表示的系统函数

$$H(s) = \frac{b_m s^m + \dots + b_1 s + b_0}{a_n s^n + \dots + a_1 s + a_0}.$$

之后，程序在同一套分析函数中完成频率响应、零极点、阶跃响应、冲激响应以及自定义输入响应的计算，保证不同输入方式得到的结果具有一致的数据结构。

Listing 1 主计算流程

```
1 params = readParams(app, app.selectedId);
2 freq = readFrequencyOptions(app);
3 model = createFilter(app.selectedId, params);
4 result = analyzeFilter(model, "plot", false, ...
5     "fmin", freq.fmin, "fmax", freq.fmax);
6 openOutputWindow(model, result, titleText, theme);
```

最终输出部分再将计算结果以绘图、数学表达式两种形式在输出的窗口中显示出来。

整体程序框架如图2：

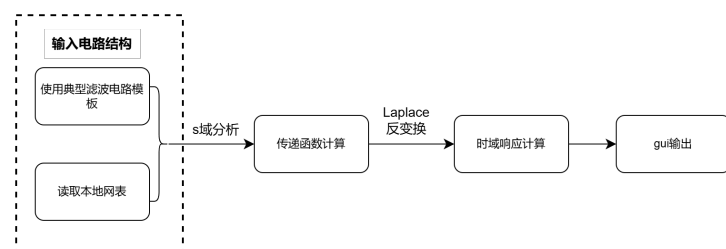


图2 整体程序框架设计

2.2.2 典型模型参数读取与传递函数计算

典型滤波电路模型的电路拓扑结构固定，因此可采用事先将模型编号、参数需求与传递函数计算公式存储在程序中的方式；目前项目中包含了无源滤波电路（一阶 RC 滤波电路、二阶 RLC 滤波电路）以及典

型有源滤波电路（二阶 Sallen-Key 滤波电路等）等常见模型。

具体模型的选择上，人机交互界面采用 html 文件较为美观地制作了不同滤波器模型的“卡片”，界面中每一张滤波器卡片对应 Matlab 中唯一的模型编号，如一阶 RC 低通对应 `rc_lowpass`，二阶 Sallen-Key 低通对应 `sk_lowpass_2`。用户选择卡片后，程序根据该模型所需的元器件名称开放相应输入框，并把输入值整理为结构体传入传递函数生成函数。需要说明的是，典型模型部分并不是根据电路图片自动列写节点方程，而是将已经推导好的公式作为模型库的一部分保存；程序运行时只负责读取参数并现场生成对应的分子、分母多项式。

Listing 2 参数读取的基本形式

```
1 function params = readParams(app, filterId)
2 spec = parameterSpec(filterId);
3 params = struct();
4 for k = 1:numel(spec.names)
5     name = spec.names(k);
6     params.(name) = str2double(app.paramFields(k).
7         Value);
8 end
end
```

典型模型的传递函数生成过程本质上是公式代入。以一阶 RC 低通和一阶 RC 高通为例，二者的分母相同，均为 $RCs + 1$ ；区别在于低通输出取电容两端，高通输出取电阻两端，因此分子分别为 1 和 RCs 。程序中用系数向量表示多项式，例如 $RCs + 1$ 表示为 $[R*C, 1]$ 。

Listing 3 典型 RC 滤波器的传递函数生成

```
1 switch filterId
2     case "rc_lowpass"
3         R = params.R; C = params.C;
4         num = 1;
5         den = [R*C, 1];
6
7     case "rc_highpass"
8         R = params.R; C = params.C;
9         num = [R*C, 0];
10        den = [R*C, 1];
11 end
12 model.num = num;
13 model.den = den;
```

对二阶 RLC 和有源滤波器，程序同样根据预定义公式得到标准二阶系统的分子、分母。对于有源滤波器，理想运放只体现为增益或反馈约束，外部电阻电容网络决定主要滤波特性。这样既保留了滤波器阶数和截止频率等关键指标，又避免了引入非理想运放模型造成的额外复杂度。

2.2.3 网表解析与电路识别

典型模型方式的优点是计算稳定、交互简单，但它要求用户手动选择滤波器结构并输入参数，无法直接处理 EDA 软件导出的实际连接关系。因此程序加入了网表解析功能，可以从本地网表文件中读取 R、L、C 元件、节点和端口信息，再根据连接关系尝试构造可分析的传递函数，这更符合“电路仿真”的要求与定位。

网表读取部分首先对文本进行预处理，删除空行、注释行和控制语句，并处理 SPICE 中以 `+` 开头的续行。随后程序逐行识别元件名称：以 R、L、C 开头的行被解析为无源元件，以 V 开头的行被解析为电压源，并作为输入端口的候选信息。对于 Multisim 等软件导出的二端口子电路封装，程序还会尝试把子电路还原为单个 R、L 或 C 元件。

典型 SPICE 网表格式如代码所示。元件行依次给出元件名称、两个连接节点和参数值；节点 0 表示参考地，控制语句用于指定交流扫描范围。

Listing 4 SPICE 网表文件格式示例

```
* Second-order passive RLC low-pass filter
V1 in 0 AC 1
R1 in n1 447.2
L1 n1 out 10m
C1 out 0 100n

.ac dec 100 10 1Meg
.end
```

端口识别采用优先级策略：若网表中存在非零交流电压源，则默认其正端为输入节点；若没有电压源，则寻找名称为 `in`、`input` 或 `vin` 的网络作为输入。输出端口则优先寻找 `out`、`output` 或 `vout` 网络。这样既能适配常见 EDA 软件导出的网表，也能保留用户手动命名端口的灵活性。

Listing 5 网表端口识别逻辑

```
1 for k = 1:numel(sources)
2     if string(sources(k).type) == "V" && ...
3         (isnan(sources(k).acMag) || sources(k).acMag
4             ~= 0)
5         inputNode = string(sources(k).nPlus);
6         inputRef = string(sources(k).nMinus);
7         break;
8     end
9 end
10 idx = find(lowerNodes == "out" | ...
11            lowerNodes == "output" | ...
12            lowerNodes == "vout", 1);
```

在得到网络与元件节点关系后，程序可在 `gui` 中将网表转化为实际电路图，基本思想为：用若干条平行的竖直向下的线表示不同网络，网络间通过若干元器件相连，根据网表中的 R、L、C 元器件信息用相应的原理图符号替代，如图 3。

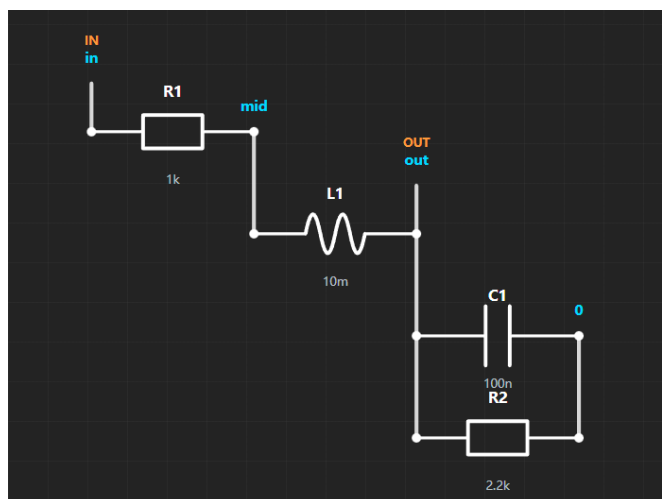


图3 网表转原理图显示

在得到电路结构后，程序进一步寻找输入端到输出端之间的串联合径，并识别输出节点到参考节点之间的并联支路，由此计算传递函数 $H(s)$ 。对于由 R、L、C 构成的梯形网络，可以用阻抗关系得到

$$H(s) = \frac{Z_p}{Z_s + Z_p},$$

其中 Z_s 为输入到输出路径上的串联阻抗， Z_p 为输出端对参考点的等效并联阻抗。

与典型模型不同，网表模式下的传递函数不是由用户选择的模型编号直接决定，而是由网表中的节点连接关系决定。程序将每个元件先转化为 s 域阻抗，再进行有理多项式的加法、求倒数和除法运算，最终得到统一的 `model.num` 和 `model.den`。因此，典型模型是“预定义公式代入参数”，网表模式则是“由连接关系现场构造传递函数”。

2.2.4 频域响应与零极点计算

频域响应的本质是将传递函数中的 s 替换为 $j\omega$ 。程序在给定频率范围内生成对数间隔的频率采样点，对每一个频率 f 令 $\omega = 2\pi f$ ，并计算

$$H(j\omega) = H(s)|_{s=j\omega}.$$

随后取 $20\log_{10}|H(j\omega)|$ 作为幅频响应，取 $\angle H(j\omega)$ 作为相频响应，并绘制波特图。该部分计算较为直接，重点在于频率范围的选择和结果的图形化显示。

Listing 6 频率响应计算

```
1 freqHz = logspace(log10(fmin), log10(fmax), npoints);
2 w = 2*pi*freqHz;
3 H = polyval(model.num, 1j*w) ./ ...
4     polyval(model.den, 1j*w);
5 mag_db = 20*log10(abs(H));
6 phase_deg = unwrap(angle(H)) * 180/pi;
```

零极点同样直接由传递函数多项式得到。若

$$H(s) = \frac{B(s)}{A(s)},$$

则 $B(s) = 0$ 的根为系统零点， $A(s) = 0$ 的根为系统极点。对于稳定滤波器，极点应位于复平面左半平面；极点越靠近虚轴，系统衰减越慢，瞬态响应持续时间越长。

2.2.5 时域响应与自定义输入

时域响应是本系统区别于普通频响绘图程序的重要部分。对线性时不变系统，冲激响应由传递函数直接反变换得到：

$$h(t) = \mathcal{L}^{-1}\{H(s)\}.$$

单位阶跃输入的拉普拉斯变换为 $1/s$ ，因此单位阶跃响应为

$$y_{\text{step}}(t) = \mathcal{L}^{-1}\left\{H(s)\frac{1}{s}\right\} = \mathcal{L}^{-1}\left\{\frac{H(s)}{s}\right\}.$$

在数值绘图时，程序将有理函数展开为部分分式。若

$$H(s) = \sum_{k=1}^n \frac{r_k}{s - p_k},$$

则其冲激响应可写为

$$h(t) = \sum_{k=1}^n r_k e^{p_k t} u(t).$$

由此可以看出，极点 p_k 不仅决定零极点图中的位置，也直接决定时域响应的衰减速度和振荡形式。

对于用户输入的一般时域信号，程序默认其右乘单位阶跃函数 $u(t)$ ，表示信号从 $t = 0$ 开始作用。设输入为 $x(t)u(t)$ ，程序先计算 $X(s)$ ：

$$X(s) = \mathcal{L}\{x(t)u(t)\}.$$

随后根据系统函数得到输出的 s 域表达式

$$Y(s) = H(s)X(s),$$

最后通过拉普拉斯逆变换得到输出时域表达式

$$y(t) = \mathcal{L}^{-1}\{Y(s)\}.$$

因此，该模块既可以输出阶跃、冲激等典型响应，也可以分析用户自定义输入信号作用下的输出结果。

Listing 7 自定义输入的符号计算

```
1 syms s t
2 Hs = poly2sym(model.num, s) / poly2sym(model.den, s);
3 x = str2sym(inputText) * heaviside(t);
4 X = simplify(laplace(x, t, s));
5 Y = simplify(Hs * X);
6 y = simplify(ilaplace(Y, s, t));
```

2.2.6 结果显示与交互输出

计算完成后，程序打开独立的输出窗口，并将结果分为波特图、时域响应、解析表达式、零极点图和结果摘要五类。波特图中同时显示幅频和相频曲线；时域响应中同时显示阶跃响应和冲激响应；解析表达式

页面使用 LaTeX 形式显示 $H(s)$ 、 $X(s)$ 、 $Y(s)$ 和 $y(t)$ 等公式，比 Matlab 原生的公式显示更加直观、美观。

这种输出方式将“可视化结果”和“数学表达式”结合起来。相比只给出仿真波形的数值工具，本系统更强调滤波电路作为信号系统的数学特征，便于从截止频率、相位变化、极点位置和时域衰减等多个角度理解电路行为。

三、实验仿真验证与结果展示

3.1 仿真器界面介绍

提交的课程设计代码的文件结构如下：

```
filter_sim/ .....项目目录
├── run.m .....GUI 主程序
├── demo.m .....快速演示
├── README.md .....使用说明
├── lib/ .....算法函数库
│   ├── filterCatalog.m .....模型目录
│   ├── createFilter.m .....构造传递函数
│   ├── analyzeFilter.m .....响应分析
│   ├── importEdaNetlist.m .....导入 EDA 网表
│   ├── parseRlcNetlist.m .....解析入口
│   └── parseSpiceValue.m .....参数换算
```

```
filter_sim/ (续)
├── lib/ (续) .....算法函数库
│   ├── reduceTransferFunction.m .....极零约简
│   └── printFilterSummary.m .....文本摘要
├── assets/ .....界面资源
│   ├── filter_cards.html .....模型卡片
│   ├── mode_switch.html .....模式切换
│   ├── netlist_viewer.html .....原理图查看
│   ├── active_cards/ .....有源电路图
│   └── btn_*.png .....按钮图标
└── examples/ .....测试样例
    ├── demo_all_filters.m .....批量演示
    ├── *.cir, *.net .....网表示例
    └── *.ms14 .....Multisim 工程
```

点击运行 `run.m` 启动程序 `gui` 界面，如图4a。

`gui` 右上角选项卡对应“典型模型”与“读取网表”两种输入方式选择，“典型模型”部分又分为无源滤波器（图4b）与有源滤波器（图4c）两大类，各自含有一阶、二阶的高通、低通、带通、带阻等常见结构；读取网表后 `gui` 程序可利用前述算法将网表转成简易的原理图（图4d）。

点击“开始计算”，程序会弹出输出窗口，其中包含滤波电路的波特图（图5a）、对 $u(t)$ 、 $\delta(t)$ 的时域响应、自定义输入的时域响应、零极点图以及结果摘要（图5b）等信息。

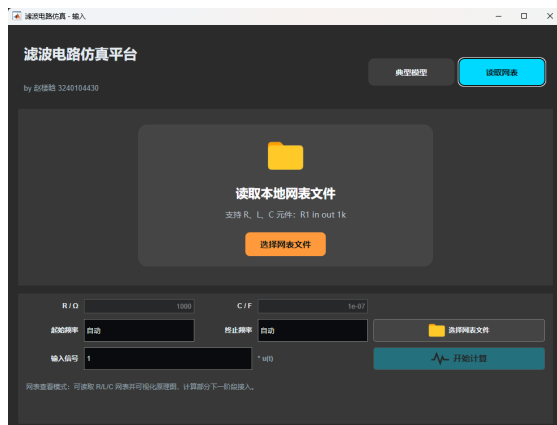


图 4(a) gui 界面

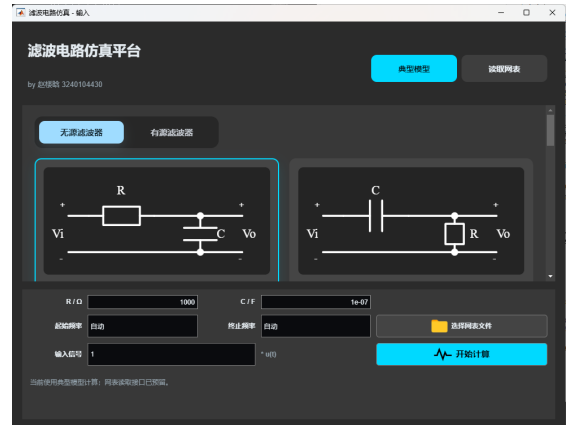


图 4(b) 典型无源滤波器选择界面

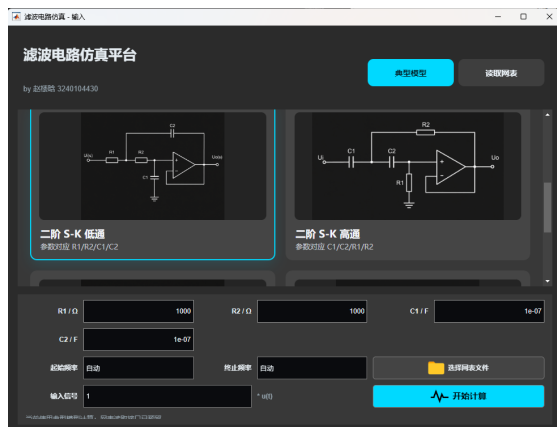


图 4(c) 典型有源滤波器选择界面



图 4(d) 网表电路可视化界面

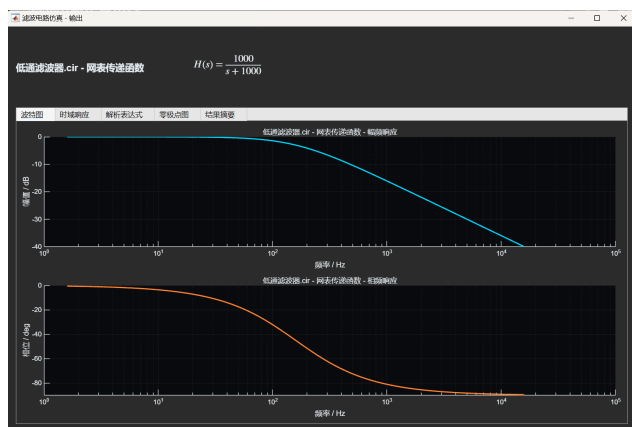


图 5(a) 波特图输出界面

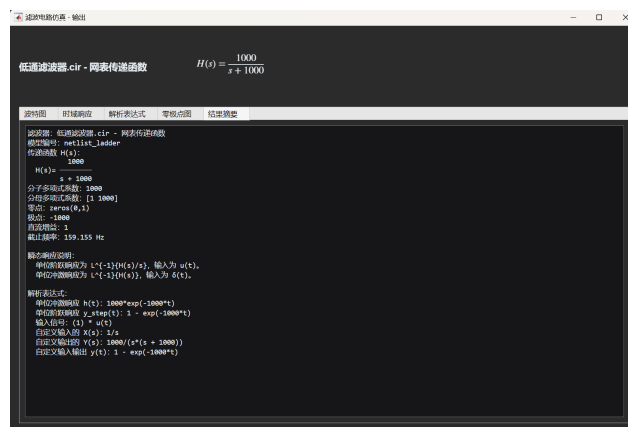


图 5(b) 结果摘要输出界面

3.2 无源电路仿真测试

使用如图6的二阶无源 RLC 电路进行测试：

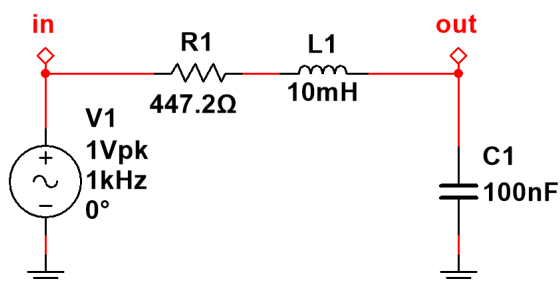


图 6 测试电路原理图

电路的理论计算如下：

输出取电容两端电压，由串联阻抗分压可得

$$H(s) = \frac{U_o(s)}{U_i(s)} = \frac{1}{LCs^2 + RCs + 1}$$

代入 $R = 447.2 \Omega$ 、 $L = 10 \text{ mH}$ 和 $C = 100 \text{ nF}$ ，得到

$$H(s) = \frac{10^9}{s^2 + 4.472 \times 10^4 s + 10^9}$$

电路的固有角频率、截止频率和品质因数分别为

$$\omega_0 = \frac{1}{\sqrt{LC}} = 3.162 \times 10^4 \text{ rad/s},$$

$$f_c \approx \frac{\omega_0}{2\pi} = 5.033 \text{ kHz},$$

$$Q = \frac{\sqrt{L/C}}{R} \approx 0.707.$$

令 $s = j\omega$ ，其幅频与相频响应为

$$|H(j\omega)| = [(1 - LC\omega^2)^2 + (RC\omega)^2]^{-1/2},$$

$$\varphi(\omega) = -\text{atan2}(RC\omega, 1 - LC\omega^2).$$

因此截止频率附近的幅值约为 -3 dB 、相位约为 -90° ，高频段幅值按 -40 dB/dec 衰减。

Multisim 仿真：

Multisim 为电子电路设计领域常用的基于 SPICE 的通用仿真软件。在 Multisim 中绘制该电路原理图，设置 10 Hz – 1 MHz 的交流仿真，得到图7的波特图，截止频率在 5 kHz 左右，可见与理论计算基本一致。

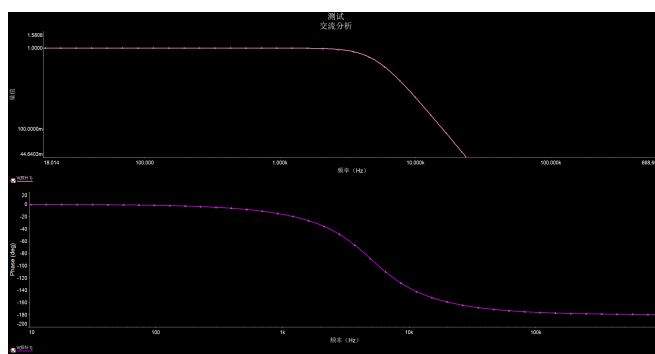


图 7 测试电路 Multisim 仿真结果

滤波电路仿真器仿真：

将 Multisim 中的原理图导出成测试 1.cir 网表文件，导入到程序 gui 中，设置输入为 $\sin(100t) \times u(t)$ ，运行仿真，得到波特图（图8a）和对输入表达式的解析（图8b）。

观察仿真结果，波特图与 Multisim 仿真所得基本一致，数学解析表达式虽较为复杂，但仔细观察也可以发现其符合正弦信号经拉普拉斯变换、逆变换后的基本特征，故可得出结论：该仿真器仿真结果较为准确。

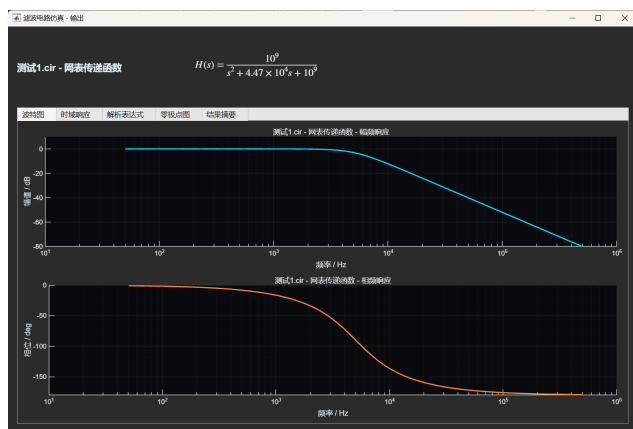


图 8(a) 测试 1 波特图输出界面



图 8(b) 测试 1 数学解析界面

3.3 有源电路仿真测试

选择仿真器中的二阶 S-K 有源带阻电路（图9）进行测试，并将运算放大器视为理想器件。取

$$R_1 = R_2 = R_4 = 10 \text{ k}\Omega, R_3 = R_5 = 5 \text{ k}\Omega, \\ C_1 = C_2 = 10 \text{ nF}, C_3 = 20 \text{ nF}.$$

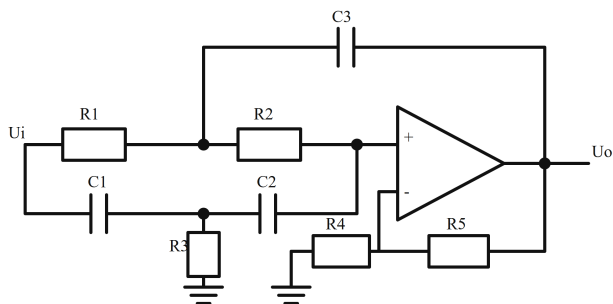


图 9 二阶 S-K 有源带阻滤波电路

电路理论计算：

设 u_a 、 u_b 、 u_c 分别为电路中三个 RC 网络节点的电压。理想运放输入电流为零，且

$$u_o = K u_b, K = 1 + \frac{R_5}{R_4} = 1.5.$$

根据基尔霍夫电流定律可列出

$$\begin{aligned} \frac{u_a - u_i}{R_1} + \frac{u_a - u_b}{R_2} + sC_3(u_a - u_o) &= 0, \\ \frac{u_b - u_a}{R_2} + sC_2(u_b - u_c) &= 0, \\ sC_1(u_c - u_i) + sC_2(u_c - u_b) + \frac{u_c}{R_3} &= 0. \end{aligned}$$

联立节点方程并代入元件参数，得到

$$H(s) = 1.5 \frac{(1 + 10^{-4}s)(1 + 10^{-8}s^2)}{(1 + 10^{-4}s)(1 + 10^{-4}s + 10^{-8}s^2)}.$$

分子、分母约去公共因子 $1 + 10^{-4}s$ 后，系统函数化为

$$H(s) = 1.5 \frac{s^2 + 10^8}{s^2 + 10^4 s + 10^8}.$$

与标准二阶带阻形式比较，可得

$$\omega_0 = 10^4 \text{ rad/s}, Q = 1, f_0 = \frac{\omega_0}{2\pi} = 1.592 \text{ kHz}.$$

其幅频响应为

$$|H(j\omega)| = 1.5 \frac{|\omega_0^2 - \omega^2|}{\sqrt{(\omega_0^2 - \omega^2)^2 + (\omega_0\omega/Q)^2}}.$$

当 $\omega = \omega_0$ 时，理想输出为零；当 $\omega \rightarrow 0$ 或 $\omega \rightarrow \infty$ 时， $|H(j\omega)| \rightarrow 1.5$ 。因此波特图应在 1.592 kHz 附近形成明显陷波，两侧通带增益约为 $20 \lg 1.5 = 3.52 \text{ dB}$ 。

仿真器仿真结果：

在输入界面中选用二阶 S-K 带阻典型结构，将 R_1 到 R_5 、 C_1 到 C_3 的数值填入，运行计算，得到图10a的波特图与图10b的瞬态响应解析表达式：

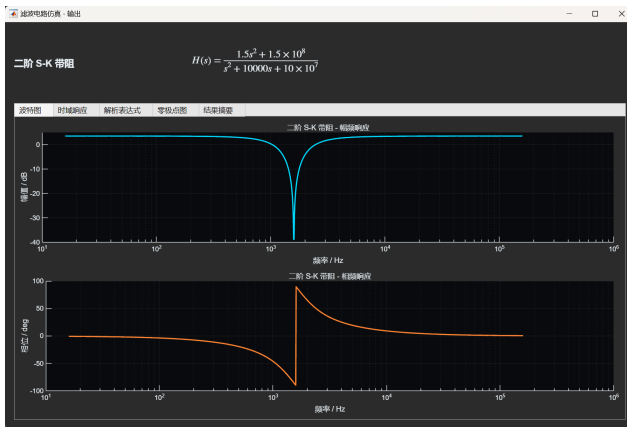


图 10(a) 测试 2 波特图输出界面

观察到幅频图中在 1.5kHz 附近有明显凹陷，对应带阻的基本形式，说明对于二阶带阻滤波电路的仿真基本正常。

四、实验结论

本实验尝试基于 Matlab 采用纯数学与信号的方式（即：拉普拉斯变换及其逆变换）构建理想滤波器的分析仿真器，从原理上不同于常见的基于 SPICE 模型的仿真软件；核心在于对电阻、电容、电感和运放等器件的理想化处理，根据公式 $Y(s) = H(s)X(s)$ 求出输出的 s 域表达式、再经逆变换求出时域的数学解析表



图 10(b) 测试 2 数学解析界面

达式。

经无源滤波电路、有源滤波电路的测试、分析，以及对理论计算结果和 Multisim 仿真的结果对比，发现仿真器的仿真结果较为准确，且响应较为迅速；此外仿真器设计了输入界面和输出界面的 gui，较为美观、直观，人机交互较为友好。

综上，本次实验较好地完成了“仿真器设计”的课程设计题目，从提出问题到提出算法到对比验证，全流程地完成了一款简易滤波电路仿真器的设计，是一次较为合格的课程设计；后续可继续增强读取网表的功能，支持更多元器件、实现更强的仿真功能。